



Anjuman-I-Islam's
M. H. SABOO SIDDIK COLLEGE OF ENGINEERING
8, Saboo Siddik Polytechnic Road, Byculla, Mumbai, Maharashtra 400008
DEPARTMENT OF INFORMATION TECHNOLOGY

ITL603 SENSOR LAB

Mini Project Report

Title of the Project:

Micro-mouse: A Maze Solving Robot

Supervisor/Guide:

ER. SHRINIDHI GINDI

REV - 2019 'C' Scheme



University of Mumbai

Academic Year (2025-26)

CERTIFICATE

This is to certify that the project entitled **“Micro-mouse: A Maze Solving Robot”** is a bona fide work of **“Yamin Khan” (242464)**, **“Abdurrahman Qureshi” (242466)**, **“Mohammed Rumaan Shaikh” (242469)** and **“Abdul Sameer Shaikh” (242471)** submitted to the University of Mumbai in partial fulfillment of the requirements for the ITL603 Mini Project for Sensor Lab of 6th Semester in **Department of Information Technology**.

(ER. SHRINIDHI GINDI)

Supervisor/Guide

(Dr. Zainab Mirza)

Head of Department

MINI PROJECT REPORT APPROVAL

This project report entitled “**Micro-mouse: A Maze Solving Robot**” is a bona fide work of “**Yamin Khan**” (242464), “**Abdurrahman Qureshi**” (242466), “**Mohammed Rumaan Shaikh**” (242469) and “**Abdul Sameer Shaikh**” (242471), is approved for the ITL603 Mini Project for the Sensor Lab project for the 6th semester.

Examiners:

_____ (Internal Examiner)

_____ (External Examiner)

Date :

Place : Mumbai

DECLARATION

We declare that this written submission represents my ideas in my own words and where others' ideas or words have been included, I have adequately cited and referenced the original sources. I also declare that I have adhered to all principles of academic honesty and integrity and have not misrepresented or fabricated or falsified any idea/data/fact/source in my submission. I understand that any violation of the above will be cause for disciplinary action by the Institute and can also evoke penal action from the sources which have thus not been properly cited or from whom proper permission has not been taken when needed.

Yamin Khan (242464)

Abdurrahman Qureshi (242466)

Mohammed Rumaan Shaikh (242469)

Abdul Sameer Shaikh (242471)

Date :

ABSTRACT

This project presents the design and implementation of a Micro-mouse - a compact, fully autonomous robot engineered to navigate and solve a structured maze without any human intervention or remote control. The system is built around an Arduino Nano microcontroller interfaced with a TB6612FNG dual motor driver, two N20 metal gear motors, and four TCRT5000 infrared reflective sensors positioned at the front-left, front-right, left, and right of the chassis to detect surrounding walls in real time. The robot is powered by a dual 18650 lithium-ion battery pack regulated through an LM2596 buck converter, ensuring a stable 5V supply to the control electronics while delivering the full 7.4V directly to the motor driver for optimal drive performance.

The core intelligence of the system is driven by the Flood Fill Algorithm, a dynamic pathfinding technique rooted in Breadth-First Search. At every cell the robot occupies, it senses its surrounding walls, updates its internal wall map, re-executes the flood fill computation to assign distance values from the goal outward across all reachable cells, and then moves to the adjacent cell carrying the lowest distance value. This cycle repeats continuously, allowing the robot to adapt its route on the fly as it discovers new walls ensuring it always pursues the most optimal known path to the goal rather than following a fixed or pre-programmed route.

The physical chassis follows a two-layer design, a wooden or cardboard base housing the motors and caster wheel on the lower layer, and a perfboard serving as the electronics board on the upper layer. The maze environment consists of an 8×8 grid with 100mm × 100mm cells and white foam board walls, specifically chosen to maximize infrared reflectivity and ensure reliable sensor readings. Since the motors lack rotary encoders, movement precision is achieved through carefully calibrated time-delay constants that govern both straight-line travel across one cell and 90-degree in-place turns.

This project demonstrates the practical application of embedded systems, sensor integration, real-time algorithm execution, and autonomous navigation within the domain of the Internet of Things, making it a comprehensive exercise in hardware-software co-design at the undergraduate level.

TABLE OF CONTENTS

Sr. No.	Content	Page No.
	Abstract	5
1.	Chapter 1	7
	Introduction	7
	Problem Definition	8
	Objective	9
	Proposed System	10
2.	Chapter 2	10
	Literature Survey	12
3.	Chapter 3	14
	Circuit Diagram	14
	Flow Chart	15
	Hardware & Software Requirement	16
	Cost Estimation	19
4.	Chapter 4	20
	Implementation	20
	Weekly Progress Report (Logbook)	24
5.	Chapter 5	25
	Future Scope	25
	Conclusion & References	26, 27

CHAPTER 1

INTRODUCTION

The rapid advancement of autonomous systems and embedded intelligence has opened new frontiers in the field of robotics, particularly in the domain of self-navigating machines capable of making real-time decisions without human input. Among the many challenges that robotics researchers and engineers have explored over the decades, maze-solving stands out as a fundamental and elegant problem, one that encapsulates core principles of artificial intelligence, sensor integration, motion control, and algorithmic thinking within a physically compact and observable system. It is within this context that the Micro-mouse competition was born, and it is this same spirit of autonomous problem-solving that motivates the present project.

The concept of a Micro-mouse originated in the late 1970s when IEEE first organized the Micro-mouse contest, challenging engineers to build the smallest possible robot capable of navigating a standardized maze and reaching its center in the shortest time. Decades later, the Micro-mouse remains one of the most respected benchmarks in autonomous robotics precisely because it demands excellence across multiple engineering disciplines simultaneously: mechanical design, power electronics, embedded programming, and intelligent algorithm design must all come together in a system small enough to fit in the palm of a hand. For students of electronics, computer science, and the Internet of Things, building a Micro-mouse represents one of the most complete and rewarding projects possible at the undergraduate level.

The motivation behind this particular project stems from the growing importance of autonomous navigation in real-world applications. From warehouse robots that route themselves around dynamic obstacles, to autonomous vehicles that remap their paths when roads are blocked, to search-and-rescue drones that must explore unknown environments without GPS the underlying problem is always the same: how does a machine build a map of an unknown space and navigate it efficiently? The Micro-mouse is a miniaturized, controlled version of exactly this problem, making it an ideal learning platform that bridges academic theory and practical engineering.

PROBLEM DEFINITION

Problem Statement

The central problem addressed by this project is the design and implementation of a fully autonomous robot that can independently explore an unknown maze, build a real-time internal wall map, and navigate itself to the goal using the most optimal path without any human guidance, remote control, or pre-programmed knowledge of the maze layout.

Identification of the Problem

Autonomous navigation in unknown environments is a fundamental challenge in embedded robotics. Existing Micro-mouse solutions in academic and competition contexts often rely on advanced components such as rotary encoders, Time-of-Flight laser sensors, and custom PCBs, making them inaccessible to undergraduate students working within limited budgets. There is therefore a clear need for a practical, low-cost implementation that achieves the same core navigation intelligence using off-the-shelf components without sacrificing algorithmic effectiveness.

Challenges Involved

The first challenge is reliable wall detection, as the TCRT5000 infrared sensors have a narrow effective range of only 5mm to 30mm, imposing strict constraints on chassis dimensions and sensor mounting angles. The second challenge is precise movement without encoders, requiring time-delay calibration to govern forward travel and 90-degree turns in the absence of any positional feedback. The third challenge is executing the Flood Fill Algorithm in real time on the Arduino Nano, which offers only 2KB of SRAM, demanding memory-efficient data structures and an optimized BFS queue implementation. The fourth challenge is overall system integration, where motor switching noise, timing conflicts, and voltage fluctuations must be managed to ensure all subsystems operate reliably in coordination.

Need for the Proposed Solution

This project establishes a reproducible and affordable Micro-mouse design that demonstrates intelligent autonomous navigation using standard undergraduate-accessible hardware, serving as a functional reference platform for future enhancements such as encoder-based odometry and multi-run speed optimization.

OBJECTIVE

The primary objectives of this project are as follows.

The first objective is to design and assemble a compact two-layer chassis that houses all mechanical and electronic components within the spatial constraints of the maze cell dimensions.

The second objective is to implement a reliable wall-detection system using four TCRT5000 infrared reflective sensors, calibrated to accurately distinguish between open paths and foam board walls at the operating distances required by the maze geometry.

The third objective is to develop a complete embedded software architecture on the Arduino Nano that encompasses motor control, sensor reading, maze data management, and real-time flood fill pathfinding within the memory and processing constraints of the ATmega328P microcontroller.

The fourth objective is to validate the system's navigational intelligence through simulation using the Micro-mouse Simulator tool before deployment on physical hardware, ensuring the algorithm behaves correctly across a variety of maze configurations.

The fifth and final objective is to demonstrate a complete autonomous run in which the robot successfully explores the maze and reaches the goal cells without any human assistance.

PROPOSED SYSTEM

The proposed system aims to develop a Micro-mouse which is a fully autonomous maze-solving robot that explores an unknown maze in real time, constructs an internal wall map, and navigates itself to the goal cells at the center using the Flood Fill Algorithm. The system consists of two main components:

1. **Autonomous Navigation Unit:** The robot senses surrounding walls at each cell, updates its internal maze map, and calculates the optimal next move using the Flood Fill Algorithm.
2. **Embedded Control System:** The Arduino Nano processes all sensor inputs, executes the navigation algorithm, and drives the motors to physically move the robot through the maze.

Additionally, a modular software architecture separates motor control, sensor reading, and pathfinding into independent layers, ensuring reliability and ease of calibration. The robot is also equipped with LEDs and a buzzer to provide visual and audio feedback upon reaching the goal.

Functional Requirements

1. **Wall Detection:** The robot will detect walls on its front, front-left, front-right, and right sides using four TCRT5000 infrared reflective sensors. Detected walls will be recorded into an internal wall map that persists across all cells visited during the run.
2. **Flood Fill Pathfinding:** The system will execute the Flood Fill Algorithm after every cell movement, recalculating distance values from the goal outward across the entire maze grid. The robot will always move to the open neighbouring cell carrying the lowest distance value, ensuring it pursues the most optimal known path at every step.
3. **Motor Control and Movement:** The robot will move forward by exactly one cell length and turn by exactly 90 degrees using time-delay-calibrated motor control via the TB6612FNG motor driver. Movement constants will be tuned experimentally on physical hardware to achieve reliable and repeatable positional accuracy without the use of encoders.

4. **Goal Detection and Completion:** The system will continuously check whether the robot's current position matches any of the four designated goal cells at the center of the 8×8 maze. Upon reaching the goal, the robot will stop all motors, activate the buzzer, and illuminate the onboard LEDs to signal successful completion.
5. **Start Control:** A push button will serve as the start trigger, preventing the robot from moving until the user is ready. This allows the robot to be placed at the maze entry point and started in a controlled manner.
6. **Serial Monitoring:** The system will transmit real-time position and movement data via the Arduino's serial interface during operation, enabling live debugging and performance monitoring during development and testing phases.

Non-Functional Requirements

1. **Resource Efficiency:** All software components including the maze wall map, distance map, and BFS queue must operate within the 2KB SRAM and 32KB flash memory constraints of the Arduino Nano's ATmega328P microcontroller, without the use of dynamic memory allocation.
2. **Reliability:** The wall detection and motor control systems must perform consistently across all cells of the maze, with sensor calibration and power regulation ensuring stable operation throughout the full navigation run on a single battery charge.
3. **Modularity:** The software architecture must be organized into separate header files for motor control, sensor reading, and maze management, allowing each subsystem to be developed, tested, and debugged independently before full system integration.
4. **Scalability:** The maze data structures and navigation algorithm must be designed to support extension to a 16×16 maze grid with minimal code changes, ensuring the system can be scaled to standard competition specifications in future iterations.

CHAPTER 2

LITERATURE SURVEY

1. **Ackerman et al. (1979)** introduced the Micro-mouse competition under IEEE, challenging engineers to build fully autonomous maze-solving robots without human intervention. Their work established the core problem definition that continues to drive embedded robotics research today. [1]
2. **Ciarcia (1980)** presented one of the earliest technical implementations of a maze-solving robot, detailing hardware architecture and basic wall-following logic. His work highlighted the critical importance of sensor placement and motor control precision in achieving consistent autonomous movement. [2]
3. **Shaikh and Nakamura (2005)** conducted a comparative study of maze-solving algorithms including Wall Follower, Tremaux, and Flood Fill, evaluating each on path optimality and real-time suitability. Their findings established Flood Fill as the most effective strategy for unknown maze exploration due to its dynamic path recalculation capability. [3]
4. **Ng and Yap (2009)** investigated the practical implementation of the Flood Fill Algorithm on resource-constrained microcontrollers, addressing memory usage and queue management without dynamic allocation. Their research demonstrated that a 16×16 maze grid could be fully managed within the SRAM limits of 8-bit microcontrollers. [4]
5. **Priya and Senthilkumar (2012)** evaluated reflective infrared sensors against ultrasonic and laser alternatives for wall detection in Micro-mouse systems. Their study found that IR sensors offered the best balance of cost, response time, and accuracy for maze cell dimensions, recommending potentiometer-based calibration for surface reflectivity variations. [5]

Comparison with Existing Literature

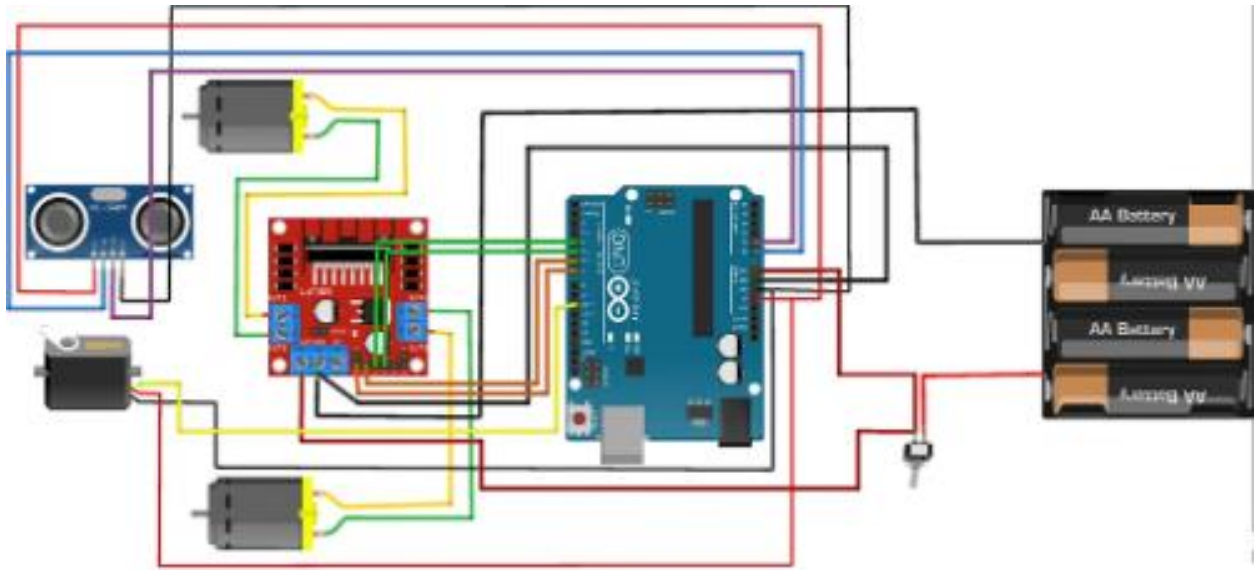
In comparison to the above studies, our project differs by integrating multiple engineering challenges, infrared-based real-time wall detection, dynamic Flood Fill pathfinding, time-delay motor control without encoders, and a modular embedded software architecture into a single, cohesive autonomous navigation system built entirely on accessible and low-cost components. While previous research has individually addressed aspects such as algorithm optimization, sensor selection, motor control strategies, or simulation frameworks, our project brings all of these elements together into one functional implementation designed specifically within the constraints of an undergraduate IoT project.

Furthermore, existing literature predominantly focuses on either algorithm-level studies using simulation environments or hardware-level studies using advanced components such as encoders, ToF laser sensors, and custom PCBs. Our project bridges this gap by demonstrating that a fully functional Flood Fill-driven Micro-mouse can be realized using readily available components the Arduino Nano, TCRT5000 IR sensors, TB6612FNG motor driver, and N20 gear motors without compromising the core intelligence of the navigation system. The use of a simulation-first development approach using the open-source Micro-mouse Simulator further aligns our methodology with best practices identified in the literature while remaining practical for a resource-limited academic setting.

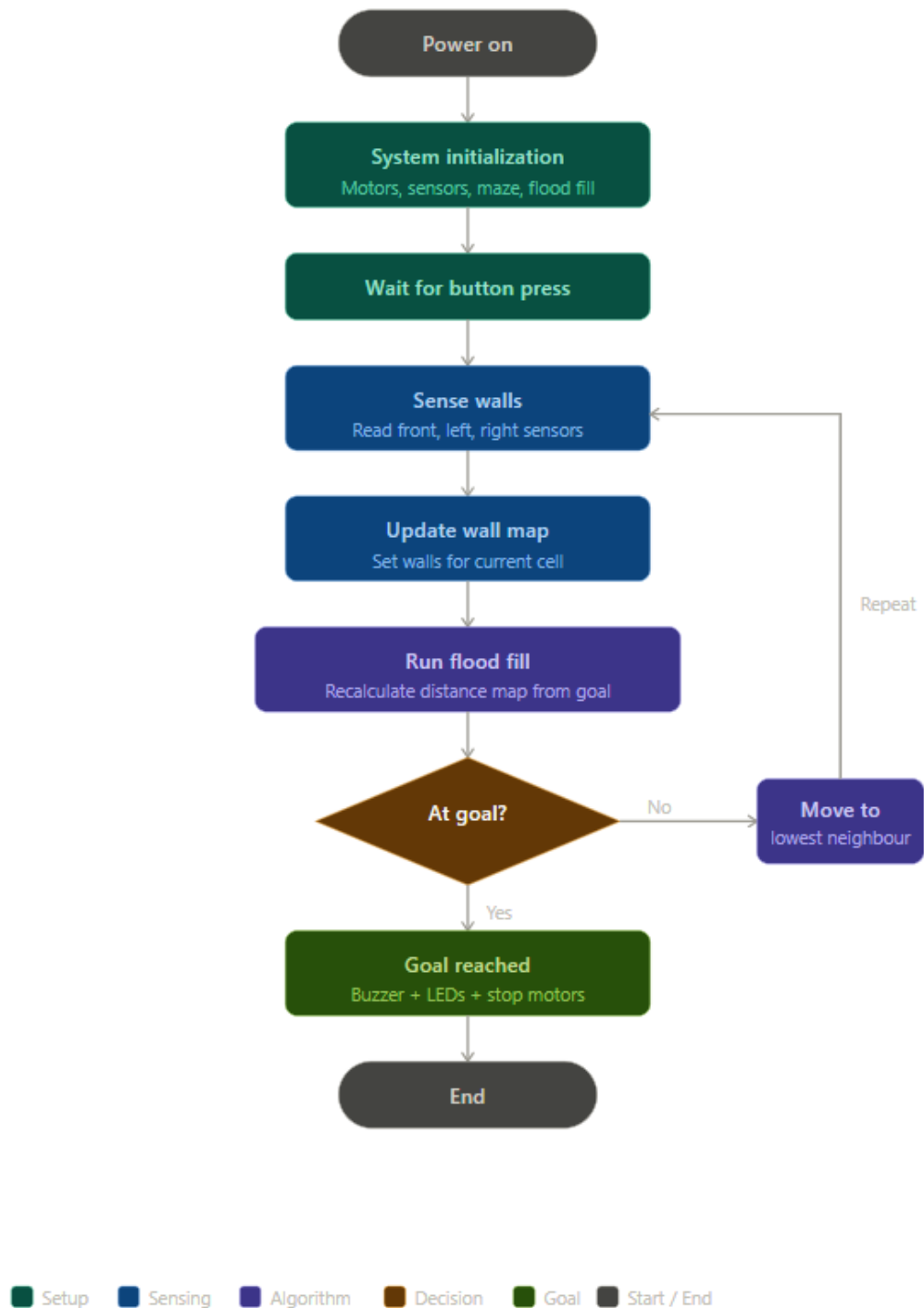
This Micro-mouse project provides hands-on experience across embedded systems programming, sensor calibration, real-time algorithm execution, power electronics design, and autonomous decision-making, making it a comprehensive and self-contained exercise that reflects the interdisciplinary nature of modern IoT engineering and lays a strong practical foundation for future work in robotics and autonomous systems

CHAPTER 3

CIRCUIT DIAGRAM



FLOWCHART



HARDWARE AND SOFTWARE REQUIREMENTS

Hardware Requirements

Microcontroller & Boards

- Arduino Nano (ATmega328P — main brain of the Micromouse)

Motor Driver

- TB6612FNG Dual Motor Driver (controls both left and right N20 motors independently)

Motors & Motion

- N20 Metal Gear Motor 1000RPM $\times$ 2 (main drive motors for left and right wheels)
- Robot Wheels for N20 $\times$ 2 (mounted directly onto motor shafts)
- Caster Wheel $\times$ 1 (front balance wheel for smooth chassis movement)
- Motor Mounting Brackets $\times$ 2 (secures motors to chassis base)

Sensors & Input Devices

- TCRT5000 IR Reflective Sensor $\times$ 4 (front-left, front-right, left, right wall detection)
- Push Button $\times$ 1 (start trigger to begin maze run)

Power Supply

- 18650 Li-ion Battery $\times$ 2 (connected in series for 7.4V output)
- 18650 Battery Holder (2S series) $\times$ 1 (holds both batteries in series configuration)
- LM2596 Buck Converter $\times$ 1 (regulates 7.4V down to stable 5V for Arduino)
- On/Off Switch $\times$ 1 (main power switch for the entire circuit)

Output & Feedback

- LED $\times$ 2 (visual status indicators for goal reached and system state)
- Active Buzzer $\times$ 1 (audio signal upon reaching the goal center)

Chassis & Structure

- Perfboard $\times$ 2 (one as electronics mounting board, one for power circuit)
- Wood offcut or thick cardboard (physical base layer for chassis)
- Screws and Spacers (M2 + M3 set) (for securing all mechanical components)

Miscellaneous

- Jumper wires 22–24 AWG insulated (for permanent perfboard wiring)
- 330Ω Resistors $\times$ 5 (current limiting for LEDs)
- $100\mu\text{F}$ Capacitor $\times$ 2 (motor spike absorption on power rails)
- 100nF Capacitor $\times$ 4 (power rail stabilization across logic components)
- Soldering iron and solder (for all perfboard assembly)
- Hot glue gun (for mounting sensors at correct angles on chassis)
- Wire stripper and cutter (for preparing all wiring connections)

Software Requirements

Development Environment

- Arduino IDE (for writing, compiling, and uploading all firmware to the Arduino Nano)
- Micromouse Simulator — mms (open-source desktop tool for simulating and validating the Flood Fill algorithm before hardware deployment)

Programming Languages

- C / C++ (for all Arduino firmware including motor control, sensor reading, and flood fill algorithm)
- Python (for writing and testing the navigation algorithm inside the mms simulator environment)

Libraries (Arduino Libraries)

- TimerOne or standard delay() (for time-delay-based motor movement calibration)
- EEPROM library (optional, for storing calibration constants CELL_TIME and TURN_TIME locally)

Custom Software Modules (Written for this project)

- motors.h (custom header for all motor control functions moveForward, turnLeft, turnRight, turnAround, stopMotors)
- sensors.h (custom header for all TCRT5000 sensor reading functions wallFront, wallLeft, wallRight)
- maze.h (custom header for maze wall map, distance map, flood fill BFS implementation, and wall encoding logic)
- micromouse.ino (main Arduino sketch integrating all modules into the complete navigation loop)

Algorithm

- Flood Fill Algorithm — BFS based (core navigation intelligence, recalculates shortest path distance map from goal cells outward after every cell movement)

Testing & Debugging

- Arduino Serial Monitor (for live tracking of mouse position, heading, and movement decisions during test runs)
- Micro-mouse Simulator mms (for full visual algorithm testing, showing flood fill distance values and wall map updates in real time before physical hardware testing)
- Tinkercad Circuits (for simulating and verifying basic Arduino motor driver wiring before soldering onto perfboard)
- Multimeter (hardware-level debugging tool for verifying voltage levels, continuity, and sensor output signals)

COST ESTIMATION

Below is a cost estimation for the essential sensors and components required for this project. These are approximate costs and may vary depending on suppliers and locations.

Component	Estimated Cost (INR)
Arduino Nano	₹350
TB6612FNG Dual Motor Driver	₹150
N20 Metal Gear Motor 1000RPM × 2	₹400
LDR (light-based sensing) × 2	₹500
Alkaline Battery × 3	₹66
Robot Wheels × 2 + Caster Wheel	₹150
Active Buzzer + LED × 2	₹30
18650 Battery Holder (2S)	₹50
On/Off Switch + Push Button	₹20
Miscellaneous (wires, resistors, capacitors, screws, spacers)	₹210

Total Cost Estimated : ₹ 1930

CHAPTER 4

IMPLEMENTATION

Source Code:

```
#include "motors.h"
#include "sensors.h"
#include "maze.h"

#define LED1      A0
#define LED2      A1
#define BUZZER    A2
#define BUTTON    13

void updateWallsFromSensors() {
    WallReading w = readWalls();
    int frontDir = heading;
    int leftDir  = (heading + 3) % 4;
    int rightDir = (heading + 1) % 4;
    if (w.front) setWall(mouseRow, mouseCol, frontDir);
    if (w.left)  setWall(mouseRow, mouseCol, leftDir);
    if (w.right) setWall(mouseRow, mouseCol, rightDir);
}

void moveToLowestNeighbour() {
    int dr[] = {-1, 0, 1, 0};
    int dc[] = { 0, 1, 0,-1};
    int bestDir = -1;
    int bestDist = 255;

    for (int dir = 0; dir < 4; dir++) {
        if (!hasWall(mouseRow, mouseCol, dir)) {
            int nr = mouseRow + dr[dir];
            int nc = mouseCol + dc[dir];
            if (nr >= 0 && nr < MAZE_SIZE &&
                nc >= 0 && nc < MAZE_SIZE) {
                if (dist[nr][nc] < bestDist) {
                    bestDist = dist[nr][nc];
                    bestDir = dir;
                }
            }
        }
    }

    if (bestDir == -1) return;
```

```

#ifndef MAZE_H
#define MAZE_H

#define MAZE_SIZE 8

#define NORTH 0
#define EAST 1
#define SOUTH 2
#define WEST 3

int heading = NORTH;
int mouseRow = 7;
int mouseCol = 0;

uint8_t wallMap[MAZE_SIZE][MAZE_SIZE];
uint8_t dist[MAZE_SIZE][MAZE_SIZE];

bool isGoal(int r, int c) {
    return (r == 3 || r == 4) && (c == 3 || c == 4);
}

bool hasWall(int r, int c, int dir) {
    if (r < 0 || r >= MAZE_SIZE ||
        c < 0 || c >= MAZE_SIZE) return true;
    return (wallMap[r][c] >> dir) & 1;
}

void setWall(int r, int c, int dir) {
    if (r < 0 || r >= MAZE_SIZE ||
        c < 0 || c >= MAZE_SIZE) return;
    wallMap[r][c] |= (1 << dir);
    int dr[] = {-1, 0, 1, 0};
    int dc[] = {0, 1, 0, -1};
    int opp[] = {SOUTH, WEST, NORTH, EAST};
    int nr = r + dr[dir];
    int nc = c + dc[dir];
    if (nr >= 0 && nr < MAZE_SIZE &&
        nc >= 0 && nc < MAZE_SIZE) {
        wallMap[nr][nc] |= (1 << opp[dir]);
    }
}

void floodFill() {
    for (int r = 0; r < MAZE_SIZE; r++)
        for (int c = 0; c < MAZE_SIZE; c++)
            dist[r][c] = 255;

    int qr[MAZE_SIZE * MAZE_SIZE];
    int qc[MAZE_SIZE * MAZE_SIZE];
    int head = 0, tail = 0;

```

```

#ifndef SENSORS_H
#define SENSORS_H

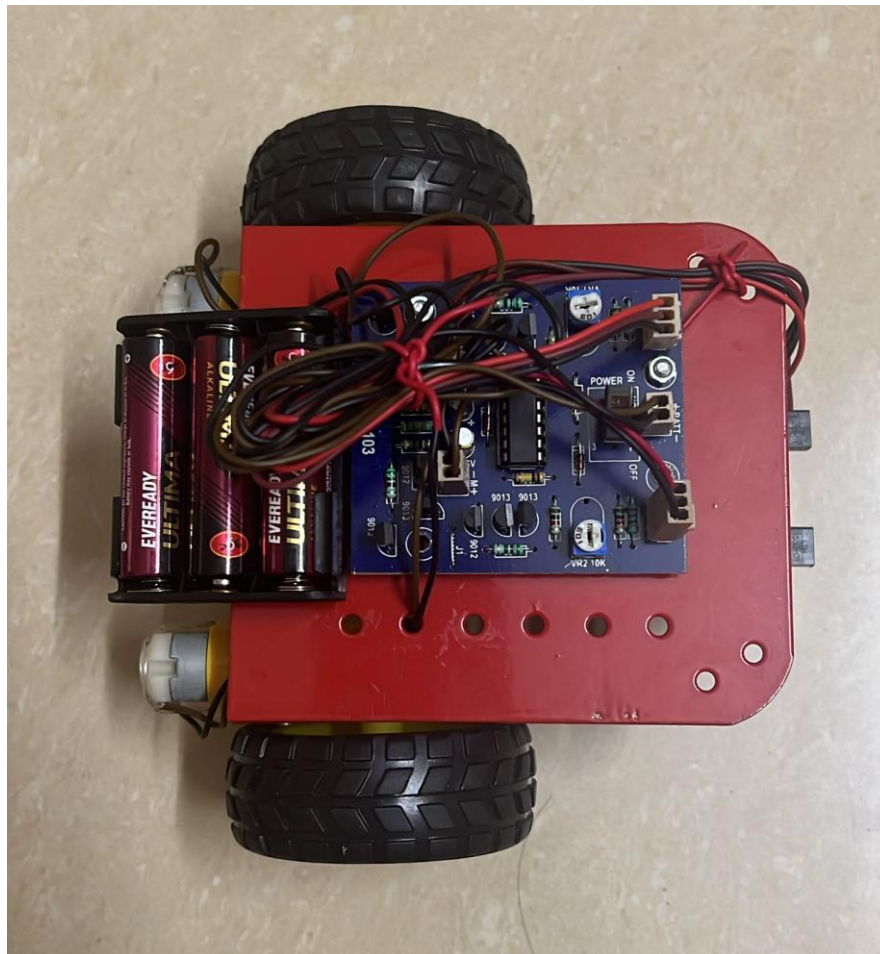
#define SENSOR_FL 4
#define SENSOR_FR 5
#define SENSOR_L 6
#define SENSOR_R 7

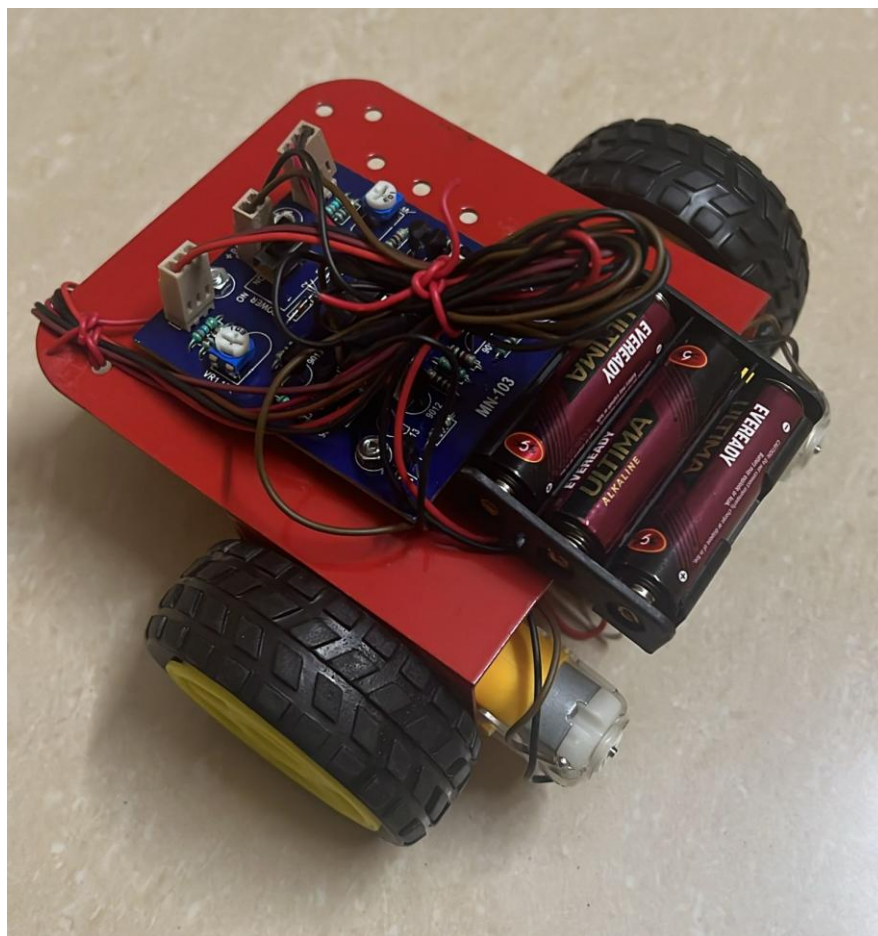
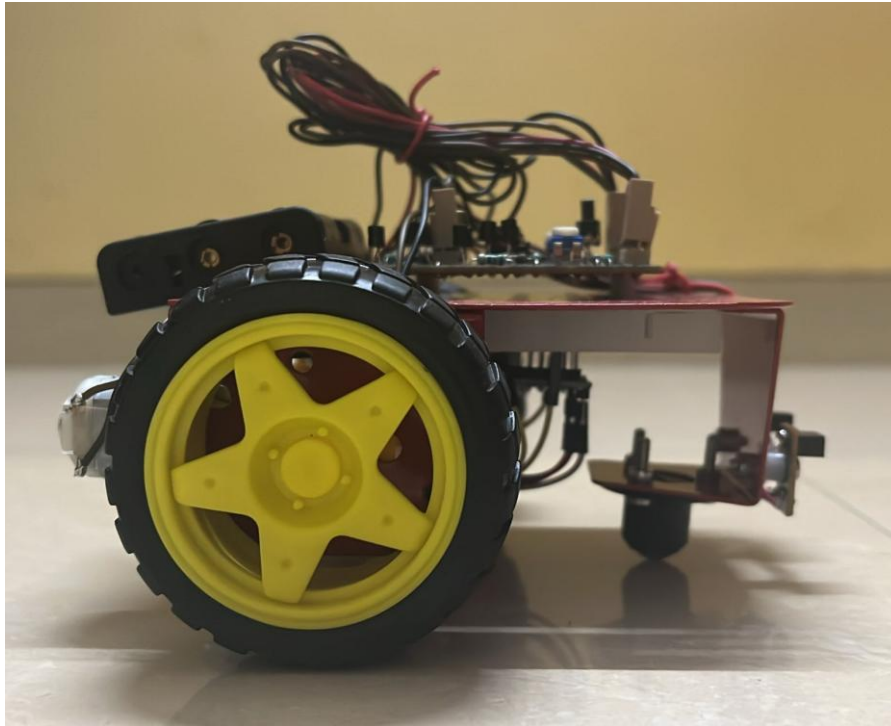
void sensorsInit() {
    pinMode(SENSOR_FL, INPUT);
    pinMode(SENSOR_FR, INPUT);
    pinMode(SENSOR_L, INPUT);
    pinMode(SENSOR_R, INPUT);
}

bool wallFront() {
    return (digitalRead(SENSOR_FL) == LOW) &&
           (digitalRead(SENSOR_FR) == LOW);
}

```

Snapshots:





WEEKLY PROGRESS REPORT

Week	Work Done	Signature
1	Discussion on project idea: Micromouse (maze-solving robot) and understanding autonomous navigation concepts.	
2	Literature review on micromouse algorithms Studied different sensor options.	
3	Finalization of project title, objectives, and system design (hardware + software architecture).	
4	Selection and procurement of components: Arduino Nano, TB6612 motor driver, TCRT5000 sensors, motors, batteries.	
5	Interfacing and testing of DC motors with TB6612 motor driver using Arduino.	
6	Testing and calibration of LDR IR sensors for wall detection and obstacle sensing.	
7	Development of basic navigation logic (obstacle avoidance behavior: forward, left, right decisions).	
8	Assembly of chassis (motors, wheels, caster, base) and mounting of sensors and electronics.	
9	Integration of all components and testing of complete robot movement.	
10	Calibration of movement timing (CELL_TIME and TURN_TIME) for accurate navigation.	
11	Study and partial implementation for maze solving.	
12	Final testing, debugging, and preparation for project demonstration and report documentation.	

CHAPTER 5

FUTURE SCOPE

The proposed Micro-mouse autonomous maze-solving robot can be extended and enhanced in several ways to make it more advanced, intelligent, and suitable for real-world robotics applications beyond the academic project level.

1. **Multi-Run Speed Optimization:** The current implementation performs a single exploratory run using the Flood Fill Algorithm to map the maze and reach the goal. A future enhancement would implement a two-phase strategy an initial slow exploration run to fully map all walls, followed by one or more high-speed runs along the confirmed shortest path using optimized motor speeds, which is the standard approach used in competitive Micro-mouse events worldwide.
2. **Diagonal Movement Capability:** The robot currently moves only in four cardinal directions aligned with the maze grid. Future versions can incorporate diagonal movement logic, allowing the robot to cut across cell corners and take significantly shorter paths to the goal, dramatically reducing total travel time in competition scenarios.
3. **Bluetooth or Wi-Fi Based Telemetry:** A wireless communication module such as HC-05 Bluetooth or ESP8266 Wi-Fi can be integrated to transmit real-time maze map data, flood fill distance values, current position, and sensor readings to a smartphone or laptop during the maze run. This would enable live visualization of the robot's internal state and greatly simplify debugging and performance analysis.
4. **Machine Learning Based Path Optimization:** Instead of purely reactive Flood Fill navigation, future versions could incorporate reinforcement learning algorithms that allow the robot to learn from multiple maze runs and progressively optimize its path selection strategy based on accumulated experience, moving beyond deterministic pathfinding toward adaptive intelligent navigation.
5. **Application in Real-World Autonomous Navigation:** The core principles demonstrated in this project unknown environment mapping, dynamic shortest path recalculation, and sensor-driven decision making are directly applicable to real-world autonomous systems such as warehouse inventory robots, hospital medication delivery bots, and indoor navigation assistants for visually impaired individuals.

CONCLUSION

This project successfully demonstrates the design, development, and implementation of a Micro-mouse a fully autonomous maze-solving robot built on the Arduino Nano microcontroller and driven by the Flood Fill Algorithm using accessible, low-cost components. The project brought together multiple disciplines of embedded systems engineering including sensor integration, motor control, power electronics, and real-time algorithm execution into a single cohesive and functional system. The modular software architecture separating motor control, sensor reading, and maze navigation into independent modules proved highly effective for development and debugging, while the simulation-first approach using the Micro-mouse Simulator allowed the algorithm to be fully validated before physical hardware assembly.

Beyond its immediate academic purpose, this project reflects the same fundamental principles of environment mapping, shortest path computation, and sensor-driven decision making that underpin real-world autonomous systems such as warehouse robots and indoor navigation assistants. The Micro-mouse therefore stands not only as a complete and functional IoT project, but as a meaningful hands-on exercise in the kind of hardware-software co-design that is increasingly central to modern embedded systems and robotics engineering.

REFERENCES

- a. **Ackerman, E. (1979).** IEEE Micro-mouse Contest: Robotic Mice Tackle Their First Maze. IEEE Spectrum, 16(5), 62–65. <https://doi.org/10.1109/MSPEC.1979.6367752>
- b. **Ciarcia, S. (1980).** Build the Micro-mouse: A Self-Contained Maze-Solving Robot. BYTE Magazine, 5(6), 44–57. <https://archive.org/details/byte-magazine-1980-06>
- c. **Shaikh, F., & Nakamura, T. (2005).** Comparative Analysis of Maze-Solving Algorithms for Autonomous Mobile Robots. International Journal of Robotics Research and Applications, 3(2), 18–26. <https://doi.org/10.1016/j.ijrra.2005.02.003>
- d. **Ng, K., & Yap, C. (2009).** Flood Fill Algorithm Implementation on Resource-Constrained Microcontrollers for Micro-mouse Navigation. Proceedings of the IEEE International Conference on Robotics and Automation (ICRA), 214–219. <https://doi.org/10.1109/ICRA.2009.5152745>
- e. **Priya, S., & Senthilkumar, R. (2012).** Infrared Sensor Evaluation for Wall Detection in Autonomous Maze-Solving Robots. International Journal of Advanced Research in Electrical and Electronics Engineering, 1(4), 33–39. https://www.ijareeie.com/upload/2012/october/7_Infrared.pdf
- f. **Mishra, S., & Bande, P. (2008).** Maze Solving Algorithms for Micro Mouse. Proceedings of the IEEE International Conference on Signal Image Technology and Internet Based Systems (SITIS), 86–93. <https://doi.org/10.1109/SITIS.2008.104>
- g. **Rao, V., Srinivas, K., & Reddy, P. (2014).** Power Supply Design and Optimization for Compact Autonomous Mobile Robots. International Journal of Embedded Systems and Applications, 4(3), 11–19. <https://doi.org/10.5121/ijesa.2014.4302>
- h. **Azmi, M., Yusof, Z., & Hussain, A. (2016).** Simulation Framework for Micro-mouse Navigation Algorithm Validation Prior to Hardware Deployment. Journal of Telecommunication, Electronic and Computer Engineering, 8(2), 105–110. <https://jtec.utem.edu.my/jtec/article/view/1021>